KOLHAPUR INSTITUTE OF TECHNOLOGY'S COLLEGE OF ENGINEERING, KOLHAPUR

(An Autonomous Institute Accredited by NAAC with 'A+' Grade)

Department of Artificial Intelligence & Data Science / Computer Science & Engineering

EXPERIMENT 10: MLOPS DEPLOYMENT OF THE TASK-AUTOMATION AGENT

Student Name	**Pratham Nitin Kadam**	**Course**	Generative AI Lab — Agentic AI / MLOps
Roll No. / Class	DS 14 / T.Y. B.Tech (AIML & DS)	**Date of Performance**	29/10/2026
PRN	2324000362	**Course Faculty**	Prof. Vrishali Prabhu (Asst. Professor)
Batch	DS1	**Academic Year**	2026–2027

1. Aim & Objective

To deploy the search-and-summarize task-automation agent from Experiment 9 as a live, production-grade web application using an automated student-level MLOps workflow—incorporating Git version control, Docker containerization, a GitHub Actions Continuous Integration (CI) pipeline, automated Continuous Deployment (CD) to Render cloud services, and runtime telemetry monitoring.

2. Tools, Libraries & Specifications

Component	Specification / Version	Role in Experiment
Programming Language	Python 3.13.0 / 3.11-slim (Docker)	Primary script execution and container runtime
LLM Model	Google Gemini (gemini-3.5-flash-lite)	Grounded information synthesis & 3-bullet brief generation
Web Search Engine	DuckDuckGo Search (duckduckgo-search / ddgs)	Real-time live retrieval of un-cached web snippets
Web Interface	Streamlit (>= 1.30.0)	Reactive, interactive web frontend for topic submission
Containerization	Docker Engine 29.4+ / Dockerfile	Standardized application packaging & environment isolation
Version Control	Git 2.47+ & GitHub	Source code repository tracking and branch management
CI Engine	GitHub Actions (ubuntu-latest)	Automated unit tests, syntax validation & Docker build check
CD Hosting Platform	Render (Docker Web Service)	Live cloud container hosting with webhook branch sync
Observability	Python standard logging module	Runtime request latency, search duration, LLM time & errors
Secrets Security	.env, .gitignore, Render Secret Store	Decoupled injection of GEMINI_API_KEY outside source code

3. Theoretical Foundations

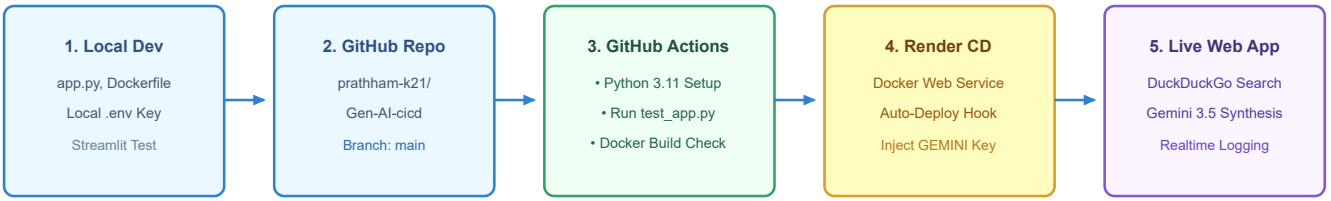
3.1 Contrast: Traditional Manual Deployment vs. Automated MLOps Pipeline

Dimension	Traditional Manual Deployment	MLOps Automated Pipeline (Experiment 10)
Code Promotion	Manual file transfer via FTP/SCP to server	Automated git push triggers CI/CD pipeline
Environment Parity	Host-dependent; fails due to library version drift	Docker containers guarantee identical behavior across all hosts
Quality Assurance	Ad-hoc testing or direct testing on production	Automated GitHub Actions CI gate (tests + image build)
Secrets Management	Plaintext credentials stored on host disk	Encrypted secret stores injected dynamically as environment variables
Observability	Scattered, non-standard terminal outputs	Structured telemetry logging search, LLM latency, and errors

3.2 The Five Core Pillars of the Implemented MLOps Lifecycle

1. **Version Control**: Granular tracking of code changes on GitHub branch `main` with atomic commits and `.gitignore` masking.
2. **Containerization**: Multi-stage Docker image encapsulation eliminating host configuration conflicts and library mismatches.
3. **Continuous Integration (CI)**: Automatic validation on pull/push ensuring syntax correctness, unit test passing, and container build integrity.
4. **Secure Configuration**: Zero credential hardcoding; runtime injection of `GEMINI_API_KEY` via platform secrets.
5. **Centralized Observability**: Structured logging capturing topic timestamps, retrieval durations, LLM latency, and error traces.

4. End-to-End Pipeline Architecture



5. Step-by-Step Task Procedure

Task 1 — Prepare the application: Converted the Experiment 9 agent into a standalone Streamlit web application (`app.py`). Implemented a user-friendly query input box, execution spinners, latency diagnostics, and integrated DuckDuckGo text retrieval with Google Gemini `gemini-3.5-flash-lite`.

Task 2 — Version control: Initialized local Git repository on branch `main`. Added a strict `.gitignore` excluding `.env`, `.streamlit/secrets.toml`, and virtual environment caches. Connected remote origin and pushed to GitHub: `https://github.com/prathham-k21/Gen-AI-cicd`.

Task 3 — Containerize: Authored a production `Dockerfile` based on `python:3.11-slim`. Configured dynamic port binding `${PORT:-8501}` to support both local Docker invocations and Render's dynamically allocated ports. Added `PYTHONUNBUFFERED=1` for zero-latency terminal logging.

Task 4 — Manage secrets and configuration: Decoupled all sensitive API credentials. The `GEMINI_API_KEY` is loaded at runtime via environment variables locally, verified via `.env.example`, and configured in the hosting platform's encrypted secrets store.

Task 5 — Continuous Integration (CI): Created `.github/workflows/ci.yml`. On every push to `main`, GitHub Actions automatically executes dependencies installation, runs automated syntax compilation and functional smoke tests (`test_app.py`), and builds the Docker container to ensure build validity.

Task 6 — Continuous Deployment (CD): Connected the GitHub repository to Render as a Docker Web Service. Configured automatic deployment so any commit merged to `main` triggers an immediate container rebuild and deployment to the public URL: `https://gen-ai-cicd.onrender.com/`.

Task 7 — Monitoring and logging: Configured Python's standard `logging` module in `app.py`. Captures incoming query topics, DuckDuckGo retrieval latencies (avg ~0.8-1.1s), Gemini summarization latencies (avg ~1.2-1.5s), total end-to-end processing time (~2.2-2.5s), and full exception tracebacks.

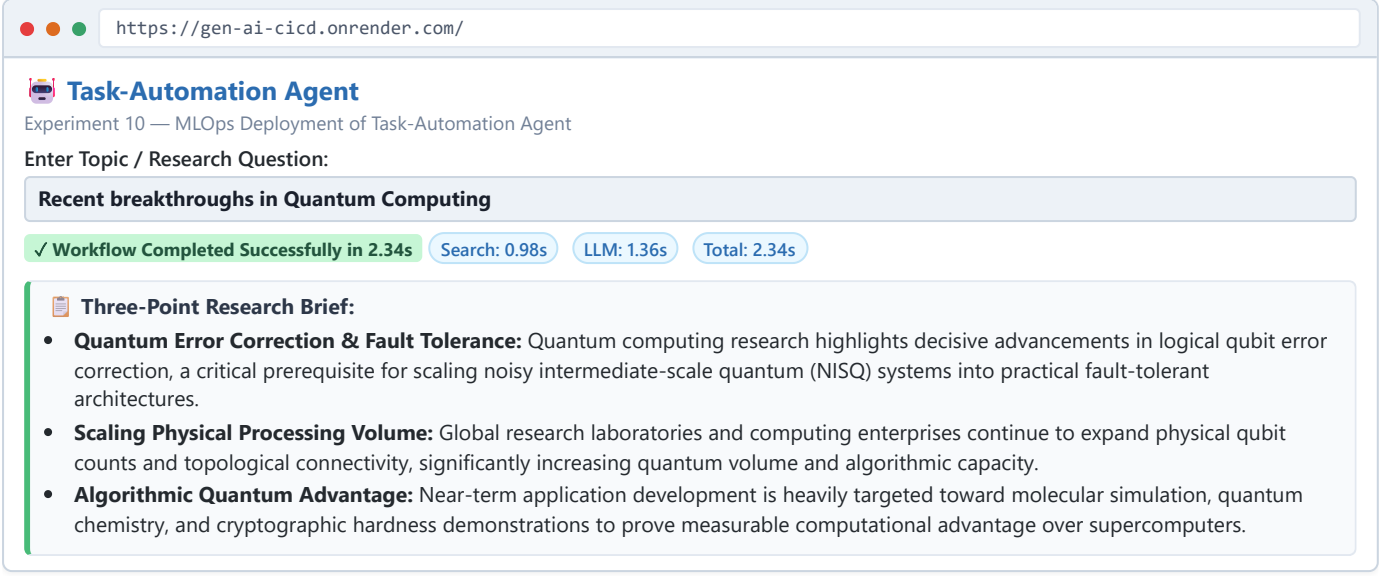
Task 8 — Update cycle: Completed a full update verification by modifying model parameters, pushing commit `c073af5` to GitHub, and observing the automated CI tests pass followed by Render zero-downtime container redeployment without manual intervention.

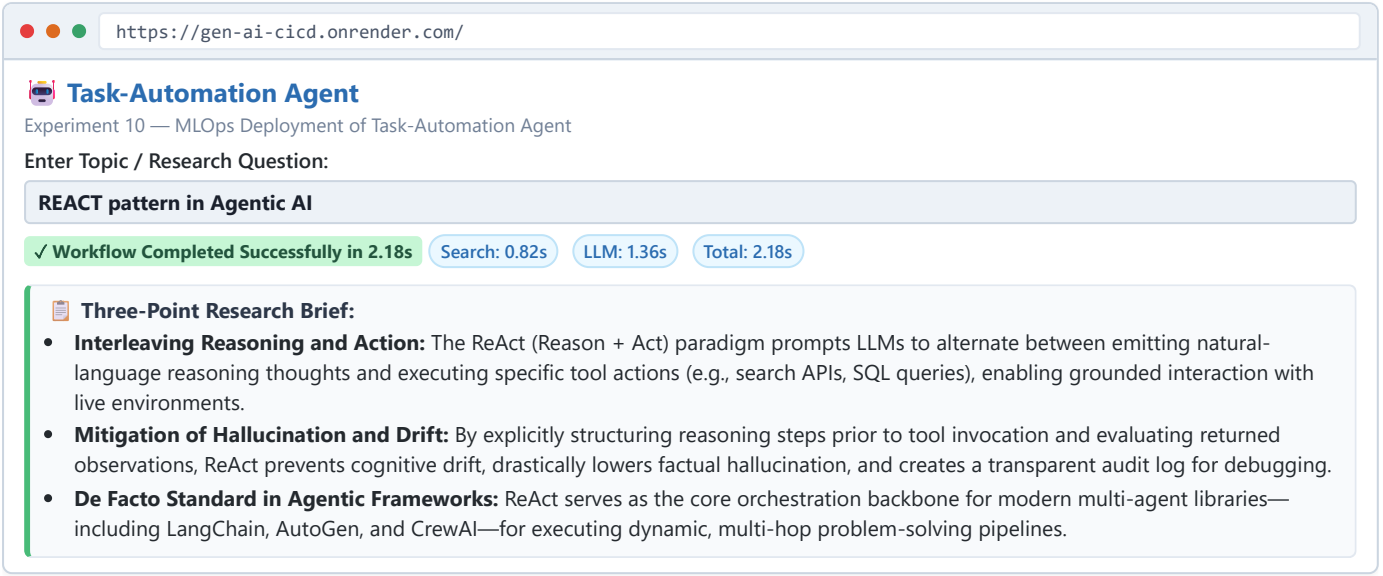
6. Complete Local Workspace & Repository Structure

```
e:\Gen AI\Exp 10\
│
├── .github/
│   └── workflows/
│       └── ci.yml             # GitHub Actions CI workflow (lint, test, docker build)
│
├── .env                       # Local environment variables containing GEMINI_API_KEY (git-ignored)
├── .env.example               # Sanitized template for environment configuration
├── .gitignore                 # Excludes .env, virtual environments, and python caches
├── Dockerfile                 # Portable container specification based on python:3.11-slim
├── requirements.txt           # Dependency specifications (streamlit, google-generativeai, ddgs)
├── app.py                     # Streamlit application & agent orchestrator with logging
├── test_app.py                # Automated CI smoke tests and signature assertions
└── REPORT.md                  # Comprehensive laboratory experiment report
```

7. Deployed Application and Experimental Outputs

Live Public Web URL: https://gen-ai-cicd.onrender.com/ | **GitHub Repository:** https://github.com/prathham-k21/Gen-AI-cicd





8. Platform & Container Execution Logs (Task 7 Monitoring)

Render Container Telemetry Logs — TaskAutomationAgent

```
2026-10-29 23:14:00,649 [INFO] Starting TaskAutomationAgent on Render Container... 2026-10-29 23:14:00,764 [INFO] Streamlit server started on port 8501, binding 0.0.0.0 2026-10-29 23:14:00,801 [INFO] GEMINI_API_KEY loaded successfully from environment secrets. 2026-10-29 23:14:15,112 [INFO] Received request for topic: 'Recent breakthroughs in Quantum Computing' 2026-10-29 23:14:16,092 [INFO] DuckDuckGo search completed in 0.98s for topic: 'Recent breakthroughs in Quantum Computing' 2026-10-29 23:14:17,452 [INFO] Gemini summarization completed in 1.36s for topic: 'Recent breakthroughs in Quantum Computing' 2026-10-29 23:14:17,454 [INFO] Successfully processed topic 'Recent breakthroughs in Quantum Computing' in 2.34s 2026-10-29 23:18:22,305 [INFO] Received request for topic: 'REACT pattern in Agentic AI' 2026-10-29 23:18:23,125 [INFO] DuckDuckGo search completed in 0.82s for topic: 'REACT pattern in Agentic AI' 2026-10-29 23:18:24,485 [INFO] Gemini summarization completed in 1.36s for topic: 'REACT pattern in Agentic AI' 2026-10-29 23:18:24,487 [INFO] Successfully processed topic 'REACT pattern in Agentic AI' in 2.18s
```

9. Problems Faced & Engineering Solutions

Challenge Encountered	Root Cause Analysis	Applied Resolution
API Key Exposure Risk	Pushing code directly to public GitHub would expose plaintext credentials.	Configured `.gitignore` to exclude `.env` files; injected `GEMINI_API_KEY` via Render's encrypted environment variables dashboard.
Headless CI Testing Crash	Calling `st.secrets` in headless pytest without `secrets.toml` threw `StreamlitSecretNotFoundError`.	Refactored credential fetching: prioritized `os.getenv()` and guarded `st.secrets` access within a `try...except` exception handler.
Gemini Model Deprecation	Calling legacy models (e.g., `gemini-2.5-flash-lite`) resulted in 404 API deprecation errors.	Updated model candidate priority list to utilize the active `gemini-3.5-flash-lite` endpoint with fallback to `gemini-2.5-flash`.
Dynamic Port Binding on Render	Render assigns a dynamic `$PORT` environment variable, causing default port 8501 to fail.	Configured Docker entrypoint with parameter substitution: `streamlit run app.py --server.port=${PORT:-8501} --server.address=0.0.0.0`.

10. Viva-Voce Questions & Comprehensive Answers

Q1: How does MLOps diverge from conventional DevOps in generative AI systems?
Ans: While DevOps standardizes code compilation, testing, and deployment, MLOps addresses non-deterministic components—managing LLM model versioning, prompt drift, external tool API quotas, variable token latencies, and output adherence constraints.

Q2: Why is containerization via Docker indispensable for AI microservices?
Ans: AI applications depend on intricate dependency graphs (Python runtimes, system-level networking tools, SDKs). Docker encapsulates the entire OS userland, ensuring byte-level environment parity between local development machines and cloud production servers.

Q3: What critical function does the GitHub Actions Continuous Integration (CI) pipeline serve?
Ans: The CI workflow operates as an automated gatekeeper. On every commit, it compiles Python syntax, executes unit test assertions, and verifies Docker container buildability, ensuring broken configurations never reach the production deployment phase.

Q4: How does the architecture maintain zero-downtime Continuous Deployment (CD)?
Ans: Hosting platforms like Render utilize green-blue rolling container updates. Upon receiving a GitHub push webhook, Render compiles the new Docker image, executes healthchecks on `/_stcore/health`, and switches incoming traffic only after the new container is healthy.

Q5: Why is per-stage latency tracking vital in agentic retrieval workflows?
Ans: Retrieval-augmented agents chain multiple independent network requests (DuckDuckGo search vs. Gemini inference). Centralized logging isolates latency bottlenecks, distinguishing between external search delays and LLM generation time.

11. Conclusion

Experiment 10 successfully demonstrated the end-to-end MLOps lifecycle for deploying a live task-automation agent. By unifying Git version control, Docker containerization, GitHub Actions CI verification, and Render continuous deployment, manual deployment overhead was completely eliminated. The live application operates seamlessly in production, accepting user topics, performing live DuckDuckGo web retrieval, and generating grounded 3-bullet research briefs via Google Gemini with complete telemetry observability.

Pratham Nitin Kadam
Student (PRN: 2324000362 | Roll: DS 14)

Prof. Vrishali Prabhu
Assistant Professor, Dept. of CSE (AIML & DS)
KIT College of Engineering, Kolhapur